

The tree: C++ and algorithms at a glance

The whole coding tutorial as one tree. Each branch is a chapter, each leaf one idea, and the tag says the chapter and section. Cover it, redraw it from memory, compare, reread the branches you missed. The structure is what you recall in the room; the words come back once the structure is there.

```
C++ AND ALGORITHMS FOR ROBOTICS
|
+-- THE LANGUAGE (enough C++ to write a solution) ch 01
|   +-- Compile and run: g++ -std=c++17 -Wall -Wextra, read the error, run the binary
|   +-- Types: int / long long / double / char / bool; integer division; overflow; casting
|   +-- Functions: by value copies, by reference (&) aliases, const& = look but don't touch
|   +-- Control flow: if / for / while, range-for, auto vs auto&
|   +-- Containers
|       |   +-- vector: push_back, size, [], 2D vector(rows, vector(cols, 0))
|       |   +-- string: substr, find, npos, to_string / stoi, the '0' trick
|       |   +-- unordered_map / unordered_set: [] inserts, find / count, auto& [k, v]
|       |   +-- pair / tuple: first / second, structured bindings, compare lexicographically
|   +-- Warm-up problems: two sum, anagram, linked list reverse / merge, Kadane,
|       |   two pointers, stock, parentheses, binary search
|
+-- MEMORY, OBJECTS, STL (what everyone asks about) ch 02
|   +-- Memory
|       |   +-- pointer = box holding an address; reference = second name on the same box
|       |   +-- const: parameters, methods, before vs after the star
|       |   +-- class: constructor, destructor, RAII (resource tied to scope), rule of zero
|       |   +-- unique_ptr (one owner) / shared_ptr (counted) / weak_ptr (observer)
|       |   +-- stack (frames, automatic) vs heap (new / delete, let containers do it)
|       |   +-- size_t is unsigned: 0 - 1 wraps; cast to int once
|   +-- STL
|       |   +-- sort + lambda comparator; strict weak ordering
|       |   +-- priority_queue: max on top; greater<> for min; pairs sort by first
|       |   +-- queue (line), stack (plates), deque (both ends)
|       |   +-- map (sorted, log n, lower_bound) vs unordered_map (hash, O(1)); iterators
|       |   +-- array (fixed size), string_view (non-owning window)
|
+-- GRIDS (the robot's map) ch 03
|   +-- grid[r][c], rows / cols, dr / dc arrays, bounds check first
|   +-- BFS = the ripple: queue, mark on push, shortest path in steps
|       |   +-- rings: snapshot queue size -> minutes / levels
|       |   +-- multi-source: push all starts -> distance to nearest (distance transform)
|   +-- DFS = walk until stuck: recursion or explicit stack; reachability, islands
|   +-- Backtracking = DFS with unmark (word search)
|   +-- Grid DP = fill a table from above and left (paths, min path sum)
|   +-- A* = Dijkstra + heuristic: f = g + h, admissible h, priority queue, lazy deletion
|       |   +-- 4-conn: Manhattan; 8-conn: octile, diagonal costs sqrt 2, no corner cutting
|
+-- GRAPHS (cities and roads) ch 04
|   +-- adjacency list: vector<vector<int>>, weighted vector<vector<pair>>
|   +-- BFS / DFS on graphs: visited array, components, cycle detection
|   +-- topological sort: Kahn (in-degree + queue); leftover nodes = cycle
|   +-- union-find: find with compression, union by size; near-constant
|   +-- Dijkstra: dist = INF, min-heap of (dist, node), skip stale, relax; non-negative only
|       |   +-- parent array -> path; early exit at target; INF = max / 4
|   +-- variants: minimax (max edge), k stops (Bellman-Ford k+1 rounds), 0-1 BFS, A*
|
+-- HEAPS, TREES, DESIGN ch 05
|   +-- heap: array tree, children 2i+1 / 2i+2, sift up / down, log n
|   +-- top-k: heap of size k (n log k) or nth_element (n)
|   +-- monotonic deque (sliding window max) / monotonic stack (next greater)
|   +-- binary tree: TreeNode, pre / in / post order, level order with ring loop
|   +-- lowest common ancestor: ask both children, report what you found
|   +-- LRU cache: hash map + doubly linked list, splice to front, O(1)
|
+-- ROBOTICS BUILDS ch 06
|   +-- Kalman filter: predict (grow P), update (blend by gain K), Q = trust model, R = trust sensor
```

```
|   +-- matrix form: position + velocity; velocity recovered from position only
+-- PID: P now, I past, D future; windup and its fixes; derivative on measurement
+-- Rotations:  $R^T R = I$ , det +1; Euler (gimbal lock) vs quaternion vs matrix
|   +-- frames:  $T_a_b$  maps b -> a; chain right to left; inverse
+-- Geometry: closest pair (divide and conquer), least squares, RANSAC (sample, count inliers)
```

The same tree in six lines

1. **Language:** compile, types, functions, control flow, four containers.
2. **Memory and STL:** pointers, const, RAII, smart pointers, stack vs heap, `size_t`; sort, heap, queues, maps.
3. **Grids:** index, BFS ripple (rings, multi-source), DFS, backtracking, DP, A^* .
4. **Graphs:** adjacency list, BFS/DFS, topo sort, union-find, Dijkstra and variants.
5. **Heaps and trees:** heap, top-k, monotonic structures, traversals, LCA, LRU.
6. **Robotics:** Kalman, PID, rotations and frames, RANSAC.

Drill

Blank page, ten minutes, redraw. Then pick one leaf at random and explain it out loud for one minute using only the tree as your notes.